

Topic W2: Decentralized Point-of-Interest Recommender System

JONAS EGGERT* and MATTES KRAUS*, Technical University of Munich, Germany

Additional Key Words and Phrases: Recommender Systems, Decentralized, Federated Learning, Gossip Learning

1 Introduction

The rise of personalized systems has led to a growing number of Location-Based Social Networks (LSBNs) such as Foursquare and Brightkite [18]. With that, the amount of publicly available data also increases, leading to new possibilities for recommending interesting places to users and improving location-based, personalized services [17, 19]. A common approach to do so is to predict the next Point-of-Interest (PoI), mostly based on a trajectory of preceding visits with timestamps, so-called check-ins, as in [22] or [40]. Classical approaches gather data points on a server to train a centralized model, capturing patterns across users [9, 18, 36]. This leads to excellent performances; nevertheless, it poses a risk to the user [32]. They draw attention to data brokers who combine check-ins with semantic data from different sources, such as your social media profile, thereby exposing sensitive information like your hobbies and interests. This can be sold to targeted advertisements, misused by insurance companies, or used to influence elections [26, 32].

An approach to address the privacy concerns is decentralized Recommender System (RecSys), especially in mobile scenarios [24]. User data and preferences stay on the device. The core idea is to separate personalization from recommendation: a local model on the smartphone captures the user’s preferences, and recommendations are then computed on the device by combining this local model with a global item model. This follows the principle of Federated Learning (FL), where a shared model is improved from many users’ data without that raw data ever leaving their devices [27].

The concrete scenario for this project is a RecSys for tourists exploring a city who proactively receive notifications about nearby PoI matching their preferences. The goal of this project is to implement a prototype application that puts this concept into practice.

Our report is structured as follows: first, we examine existing decentralized RecSys architectures (Section 2). Second, we introduce our methodology: the formulation of our problem, the dataset we train on, and the approaches we use to solve it (Section 3). In Section 4, we present our frontend prototype that integrates the models and the freedoms the user has using it. In Section 5, we evaluate the performance of the models. Lastly, in Section 6 we provide a brief outlook on potential future work, and finally, we summarize our conclusions for the project in Section 7.

2 Related Work

The main goal of this project is to keep the user’s sensitive data private while still recommending personalized PoIs (Section 1). To guarantee that personal data doesn’t leave the user’s device, various approaches have been proposed in the distributed recommender system domain [3]. The common idea is to distribute either the training, the inference, or both. A figure visualizing this categorization is shown in the appendix (Fig. 7).

While approaches such as Collaborative Inference [38] and *Distributed Task Collaboration* [34] are prominent in this domain, they are not suitable for our scenario. Collaborative Inference distributes the prediction process across devices, preserving privacy for non-sensitive inputs but failing when both the model and the input (e.g., location) are sensitive. Distributed Task Collaboration, on the other hand, relies on completely independent

*Both authors contributed equally to this research.

training and inference on each device. While privacy-preserving, it prevents the system from learning from other users' experiences, limiting recommendation quality and scalability.

To overcome these limitations and profit from shared knowledge without compromising privacy, we focus on Collaborative Learning. Here, inference occurs independently on the device, while the model is trained collaboratively [3]. This perfectly fits our scenario. In the following, we examine two prominent architectures in this domain: Device-Cloud and Decentralized Collaborative Learning.

2.1 Device-Cloud Collaborative Learning

The idea of device-cloud collaborative learning is depicted in Fig. 1. The intuition is similar to updating your centralized model with gradient descent. You have a model; calculate the gradient from a data sample and update the model parameters based on a learning rate times the gradient [37].

$$\theta_{new} = \theta_{old} - \tau \nabla \mathcal{L}(x|\theta_{old})$$

where θ are the model parameters, τ is a fixed learning rate and $\mathcal{L}(x)$ is the loss function for a specific data point [4].

In classical stochastic gradient descent, the algorithm has access to all data points, which, in practice, leads to the simplest way to store the data in the cloud [4]. This exposes the user to a risk: the data is stored in the cloud and must be stored there, leaving the device vulnerable to various attacks [30]. Therefore, instead of calculating the gradient in the cloud using centralized data, device-cloud collaborative learning methods bring the model to the device and let it compute the gradient from its local data [37]. To protect users against feature-inversion attacks, in which attackers reconstruct the original training data from the model's initial parameters and gradients, one could add noise to the gradient [25, 30]. The model updates its parameters based on the gradient and behaves as if it had been trained on that user's data. The advantage is that the actual data was never exposed outside the user's device, thereby preserving the user's privacy [3, 37].

2.2 Decentralized Collaborative Learning

The idea behind decentralized collaborative learning algorithms is similar to that of device-cloud collaborative learning. Instead of actual data points, the models only exchange gradients, which they use to update themselves [3]. The process is depicted in Fig. 1. Nevertheless, as device-cloud collaboration can be seen as a tradeoff between decentralized data and a centralized model, decentralized collaborative learning is fully decentralized and operates entirely without a cloud. The local models exchange their gradients directly with each other [39]. Even with a varying graph topology, it can be proven that, given sufficient information exchange, the local models will converge to a global optimum [15, 20].

3 Methodology

3.1 What to Predict

Before we selected a model architecture for the RecSys, we had to define the prediction target itself. In LSBNs like Foursquare the fundamental user action is the check-in. This occurs when a user announces their presence at a PoI at a specific time [36]. A PoI can be any visitable place such as a museum, park or train station. Formally a check-in is represented as a tuple [31]:

$$c = (u, p, t), \tag{1}$$

where $u \in \mathcal{U}$ is a user, $p \in \mathcal{P}$ is a PoI associated with GPS coordinates and a venue category, and t is a timestamp. Ordering all check-ins of a user u by time give their check-in sequence (or trajectory)[31]:

$$S_u = (c_1, c_2, \dots, c_n), \quad t_1 < t_2 < \dots < t_n. \tag{2}$$

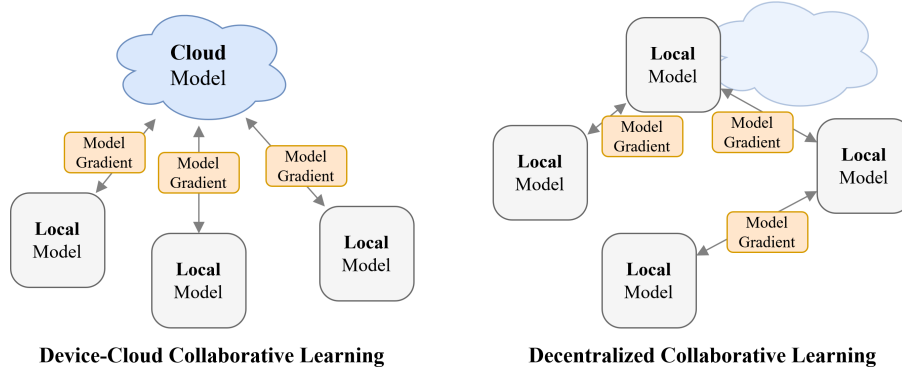


Fig. 1. A visualization of device cloud and decentralized collaborative learning as in [3]. On the left, you can see the local models exchanging model gradients with a centralized cloud model. On the right, the local models share model gradients only with each other, entirely without a centralized model.

We frame our approach around sequential next PoI prediction rather than a rating prediction which also influence our dataset choice. Check-ins offer purely implicit feedback, unlike explicit ratings which suffer from severe data sparsity because most users rarely rate venues [5]. Also sequences capture crucial spatiotemporal patterns which a only rating based approach discard [41]. This makes sequence based modeling better suited to capturing how user preferences actually unfold in real world behavior.

3.2 Dataset

For all of our experiments, we chose the New York City (NYC) part of the Foursquare check-in dataset released by Yang et al. [36]. It contains check-ins collected in NYC and Tokyo (TKY) between April 2012 and February 2013 from the LSBNs Foursquare. Each check-in is an individual time-stamped visit that a user made to a venue. It comes with the venue’s GPS coordinates and a semantic category (e.g., cafe, museum, train station). The data forms a sequence of visited PoIs per user, which is exactly what we need for our sequence model to learn from. This motivated our choice over the alternatives we considered. The Yelp Open Dataset, for instance, is built around explicit feedback (star ratings and written reviews) but only exposes check-ins as aggregated hourly counts, since the actual visit order for the users is lost, it does not support sequence modeling [6]. The Foursquare NYC/TKY dataset has also become a standard benchmark for next-PoI recommendation and is used across a wide range of recent work, making our results comparable [1, 33, 42].

Concretely, the NYC file contains 227,428 check-ins and the TKY file 573,703, each stored as a tab-separated row with the fields shown in Table 1. A trajectory is the ordered list of such rows from a user, which we later turn into the input sequences.

As indicated in the table, the semantic venue categories are not flat but follow the hierarchical Foursquare Category Tree. This taxonomy ranges from nine broad Level-1 categories (e.g., *Dining & Drinking* or *Arts & Entertainment*) down to highly specific Level-4 subcategories (>100 different categories, e.g., *Asian Restaurant*).

3.3 Category based Approach

As we saw in Section 3.2, we have limited access to multiple check-ins directly after each other, but more like a global picture of single check-ins per user. With that, it is difficult to find patterns, like after going to a restaurant at midday, they always go shopping. Therefore, we wanted to develop an approach that can be trained without finding patterns in the exact sequence but can extract them from different features.

Field	Example
User ID	470
Venue ID	49bbd6c0f964a520f4531fe3
Venue category	Sushi Restaurant
– Level 1 Category	Dining & Drinking
⋮	⋮
– Level 4 Category	Asian Restaurant
Latitude	40.71981
Longitude	−73.99326
Timezone offset (min)	−240
UTC timestamp	Tue Apr 03 18:00:09 +0000 2012

Table 1. Structure of one check-in in the Foursquare NYC/TKY dataset, including the hierarchical category mapping.

Wiedemann et al. [32] did the opposite of what we are trying; nevertheless, it is a very interesting approach we can use for our scenario: they tried to *attack* user privacy. Based on a location obfuscated by different radii, they tried to predict the place’s category. Just from location data, they can determine which types of locations the user likes and then create so-called soft profiles, which cluster users. With that, they wanted to show that sensitive data, such as location histories or visit categories, should never be stored in a centralized database.

But we can also learn from that that their approach to predicting categories of places users often visit based on location and similar users works! Therefore, we can try a similar approach, and instead of predicting the exact next-PoI of the tourist, we predict the next-PoI category. Nevertheless, we model the architecture differently and thereby guarantee user privacy.

3.3.1 Baseline and its Limitations. Wiedemann et al. [32] uses a centralized approach to predict a location’s category based on its coordinates, obfuscated by a radius ranging from 0 to 1200m, temporal features such as check-in time, and spatial features, for example, indicating which other categories are nearby. They use an XGBoost model, which they do not specify further, and the same Foursquare dataset we use. They reach an accuracy of 100% when they do not obfuscate the location at all. This is expected, as the model can learn the exact category per place. The performance decreases with increasing radius, down to 20% with a radius of 1200m. The model, which has access only to the temporal data and not to the coordinates or other spatial features, achieves 29.7% accuracy on the NYC dataset.

Nevertheless, they did not put their code online, so the only information we have is what they describe in their paper. Based on that, from our perspective, the spatial features are not feasible for our tourist scenario; in the end, this is a design question. Should the recommender system propose the best place from the places in the vicinity, or should it predict good categories at the current time, regardless of whether you would have to walk a bit further? Using spatial features, the model will learn which category from the PoIs around the user is the best fit. We think this is not what the tourist recommender system should do. If it is 8 p.m. and the available categories around you are a café and a swimming pool, you should take the tram and drive to a restaurant or a bar. Therefore, we chose to only implement the temporal features.

Based on users’ category visits, Wiedemann et al. [32] also developed features to cluster users into groups. With that, they created a profiling tool that could be sold to companies or misused in other ways. But the features proposed in their work are pretty sparse. They take the nine broad level-1 categories from all check-ins of a user and build the distribution of them. A user with three check-ins in Retail and one check-in in Nightlife would

yield a profile of:

$$user_x = [\underbrace{0.75}_{\text{Retail}}, \underbrace{0.25}_{\text{Nightlife}}, \underbrace{0, \dots, 0}_{\text{other categories}}]$$

This is a good start but from our perspective for a clustering of >1000 users too little.

3.3.2 Our model (centralized). In the following, we will explain how our recommendation model generally works and what it is based on. You can imagine it as a centralized model that has access to all the data. In the next section (Section 3.3.3), we will discuss the transition to an entirely decentralized approach.

Again, we take the idea of profiling and modify it. Because we want to predict the best fitting categories for the tourists, we create a similar profiling. Instead of attacking users' privacy, we adjust recommendations based on each user's profile. Because the baseline features were too little, we introduced an additional six features per user, as shown in Table 2.

	Category Pref.	Time Preference	Weekend Ratio	Exploration Rate
Logic	Top-9 Level-1 category distribution.	Check-in frequency per time segment.	$\frac{\text{weekend check-ins}}{\text{total check-ins}}$	$\frac{\text{unique venues}}{\text{total check-ins}}$
Features	9 columns	4 columns	1 column	1 column
Range	$x_i \in [0, 1]; \sum_{i=1}^9 x_i = 1$	$t_j \in [0, 1]; \sum_{j=1}^4 t_j = 1$	$w \in [0, 1]$	$e \in [0, 1]$

Table 2. Extracted user features for K-Means clustering.

Using a (first, you can imagine it as centralized) k-means clustering, we extracted 20 clusters [21]. For a better understanding, we added three visualizations of them in Appendix A.2. The idea of k-means clustering is as follows: as a first step, we build user features for all users, so we have all the necessary data. Next, we randomly initialize k centroids (in our case, 20). They are points in the same space as our data lies. For each user datapoint, we calculate which centroid is the closest. With that, we get an ordering of the datapoints. As the last step, we update the centroids by taking the mean of all datapoints in each cluster and replacing each centroid with that mean. By iteratively doing that, the centroids eventually converge and stop changing. As a result, you store those centroids. If we want to classify new datapoints, for example, because a new user registered, we calculate the distance to each centroid and assign the user the cluster of the closest centroid. With that, we now have a way of clustering users based on their check-in behavior. Next, we want to predict each user's categories.

As discussed in Section 3.3.1, we want to predict the category of the next PoI based on temporal features and check-ins of similar users. With that, we propose the following model architecture, depicted in Fig. 2.

Using the centroids to cluster the users, we have a clear group for each recorded check-in. Based on that, we can determine which category was popular in each user cluster at a given time. This is what we call the *popularity score*. We do this count for each check-in we have access to. As a result, we obtain a database of popularity scores for each cluster at each time of day. If a user now wants a recommendation on where they should go next, based on their past check-ins and the trained centroids, we obtain their cluster. Then, using the cluster number and time, we predict the top k most popular categories within his cluster.

3.3.3 Our model (decentralized). Given the model architecture proposed in Section 3.3.2, we would store the popularity score in a centralized database. In practice, each user would have to send their check-ins along with their user-cluster to the cloud. That would expose immense risk to their sensitive data location data [32]. Therefore, we propose an entirely decentralized approach to train k-means for user clustering and to track popularity scores across devices.

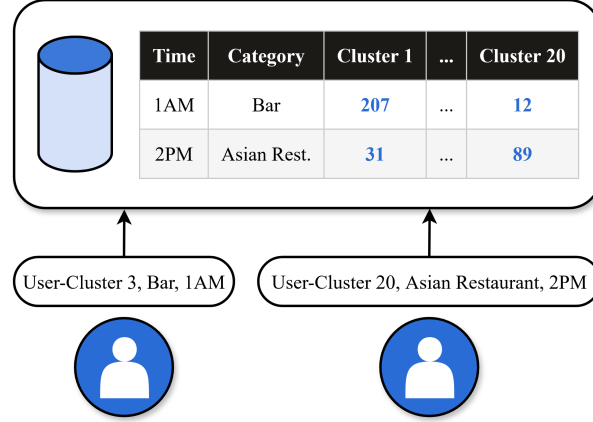


Fig. 2. The idea of the category prediction model. We keep track of which user cluster visited which categories at what time. With that, we get what we call *popularity score*, which is the sum of all check-ins in that specific category at that specific time. In this case, 207 people from user group 1 went to a bar at 1 a.m., and 31 went to an Asian restaurant at 2 p.m.. From user group 20, there were only 12 visits to a bar at 1 a.m., but 89 to Asian restaurants at 2 p.m.

For the user model, we considered the paper by Fellus et al. [8], which proposes a decentralized k-means approach. The core idea is that the algorithm runs in epochs, with users performing multiple gossip rounds within each epoch. In a round, users group up in random pairs and share their current beliefs about the k centroids (sharing their accumulated centroid sums and counts). They average these values, meaning that after their exchange, both peers share the exact same updated belief. In the next round, they pair up with new peers, repeating this process until the local centroids converge across the network. The main advantage is that data is never centralized on a server. Furthermore, an individual’s pure data is exposed to at most one random peer during the very first round. After that initial exchange, the data becomes mathematically blended and untraceable. From then on, everybody only shares aggregated means, ensuring that no raw datapoints are ever stored or shared with other users.

With that, every user has access to their user centroids, enabling them to determine which global cluster they belong to. Next, we need to decentralize the popularity scores. For that, we use Gossip Learning as in Belal et al. [2], an algorithm in the decentralized collaborative-learning domain (see Fig. 1). Each user acts as an independent node that first computes histograms of their own check-in behavior and deliberately adds random statistical noise (Laplace noise) to obscure their true data, ensuring Local Differential Privacy [7]. The nodes then engage in randomized gossip rounds, pairing up to average their noisy histograms; as the network gossips, the individual noise cancels out, allowing the entire network to converge on highly accurate global popularity metrics from which local top- k recommendations can be extracted, all without a central authority.

3.4 Advanced transformer based Approach

As outlined in Section 3.3.2, the category-based approach does not include location history, and it is a rather simple model. To address this, we implemented a more advanced transformer-based approach. We will present it as follows.

3.4.1 The FedKG Framework. Our advanced recommendation model builds on FedKG, a federated next-PoI framework proposed by Zhong et al. [42]. This section introduces only the core concepts needed to understand our project, the full framework is described in the original work.

3.4.2 Federated Learning Setup. The prediction task follows the problem description in Section 3.1. Given a user’s chronologically ordered check-in sequence the goal is to predict the next visited venue. To become decentralized FedKG solves this in a federated learning setting. The data is partitioned over N clients, each containing the trajectories of the users assigned to it. Every client i trains a local copy θ_i of a shared model on its private data D_i and uploads only the parameter updates. The server aggregates them into a global model θ_g weighted by the client data sizes, following the principle of FedAvg introduced by McMahan et al. [23].

$$\theta_g = \sum_{i=1}^N \frac{|D_i|}{|D_{\text{local}}|} \tilde{\theta}_i, \quad [42] \quad (3)$$

Then θ_g is shared back for the next round. Raw check-ins never leave the device.

3.4.3 Base Recommender: SASRec. As the shared model θ we chose Self-Attentive Sequential Recommendation (SASRec) [13], a sequential recommender based on the transformer architecture [29]. It maps every PoI to an embedding, adds a positional encoding to capture order and recency, and uses stacked self-attention blocks to weigh which past check-ins are relevant for the next step. At inference it produces a relevance score for every candidate PoI, the highest-scoring venues form the Top- K recommendations. SASRec has been shown to have a better performance than convolutional sequential recommenders e.g. GRU4Rec [11] and Caser [28] while remaining efficient [13]. The lightweight design keeps the per round computation and communication cost low, making SASRec well suited for the resource constrained federated setting.

3.4.4 ST-GAN: Spatiotemporal Data Augmentation. The individual clients only holds little data in their region. To compensate for this FedKG proposes the ST-GAN a Spatiotemporal Generative Adversarial Network [42]. This is a generative adversarial network [10] tailored to spatiotemporal trajectory data. Its generator G receives three aligned input sequences per trajectory (the visited PoIs, their timestamps, and the corresponding coordinates) and learns their joint distribution in order to generate trajectories that are plausible but do not correspond to any real user. A discriminator D is trained in parallel to distinguish real from synthetic trajectories providing the learning signal that drives the generators improvement. The resulting synthetic trajectories are merged with each client’s real data. In addition the locally trained generator parameters can be averaged on the server into a global generator whose synthetic data also reflects global patterns. An also we can’t distinguish any more between real and synthetic trajectories we improve also the data protection.

3.4.5 Bidirectional Knowledge Distillation. In federated learning, averaging all client models into one global model has a drawback. The specialized knowledge each client has built up gets watered down. FedKG tackles this with knowledge distillation [12], where one model called teacher passes its knowledge to the student model by training the student to reproduce the teacher’s outputs. In FedKG, this happens in both directions. With Personalized Knowledge Distillation (PKD), the client’s previous local model teaches the incoming global model so that local spatial patterns are not lost in the update. With Global Knowledge Distillation (GKD) the roles flip. The global model teaches each local model and shares cross regional knowledge that no client could learn from its own data alone. Both are trained with the same loss function. One part rewards correct predictions on the task itself, while the other measures how closely the students’ output matches the teacher’s, using the KL divergence with a temperature parameter that softens the teacher’s predictions and thus makes them easier to learn from [12].

3.4.6 Training Loop. A single communication round chains the three modules as follows. First the server broadcasts the current global model to all clients. On the client side PKD then adapts this global model to each client by distilling knowledge from the previous local model into it, after which the client continues with standard local training. The updated local models are uploaded to the server where GKD refines them with global

knowledge before they are aggregated into the new global model. The ST-GAN is trained only once before the federated rounds.

3.4.7 Data split. Following the standard leave-one-out protocol [13], we split each user’s check-in sequence chronologically: the final check-in serves as the test item, the second-to-last as the validation item, and all prior interactions for training. While the validation set guides model selection and early stopping, the test item is evaluated only once against the entire POI catalog. This setup ensures the model is always tested on its ability to predict a future, unseen visit.

3.5 Adaptations to FedKG

Starting from the reference implementation of FedKG [42], we developed two tuned variants. One with and one without the ST-GAN module that are otherwise identical, so that the effect of the generative component can be isolated. Our modifications follow three main purposes: making the evaluation honest, aligning the training objective with it and stabilising federated optimization. Table 3 summarises them.

Aspect	Original → Ours	Why
Evaluation	100 sampled neg. → full ranking	honest, deployment-like
Model selection	test-set max → validation NDCG@10	removes selection bias
Training loss	BCE, 1 neg. → sampled softmax, 200 neg.	matches ranking task
Learning rate	0.01 fixed → 0.001 warmup+decay	stable FL training
Regularization	none → weight decay + early stopping	less overfitting
Model size	50d, 1 block → 64d, 2 blocks	more capacity
Participation	40% clients/round → 100%	every client contributes
Data cleaning	all users → drop home/office-dominated	removes low-signal data
ST-GAN	always on → optional	with/without comparison

Table 3. Modifications from the original FedKG to our tuned variants.

3.5.1 Honest Evaluation. A recommender is judged by how highly it ranks the venue the user actually visited next. The original implementation, however, compares this target venue against only $num_neg = 100$ randomly sampled wrong venues, so called sampled ranking. A much easier task than ranking it against every venue in the city, and one that is known to produce strongly optimistic metrics [16]. We therefore switched full ranking evaluation, where the target venue competes against all candidate venues, exactly as a deployed system would face. As we can see in Fig. 3 the same model that reaches $\approx 27\text{--}30\%$ NDCG@10 under sampled ranking drops to $\approx 1\%$ under full ranking.

3.5.2 Aligning the Training Objective. A model can only improve at what its loss function rewards, the training objective should therefore reflect the evaluation criterion. The original binary cross-entropy loss contrasts each visited venue with a single randomly sampled negative, reducing training to a sequence of pairwise discriminations. Succeeding at these one-versus-one comparisons does not imply that the correct venue is ranked highly against the full catalog. A sampled-softmax loss removes this mismatch by scoring the correct venue against 200 negatives jointly, so that each update optimizes the same top-of-list ranking that the metric evaluates. Sampling 200 negatives rather than the full catalog keeps the loss computationally tractable [35]. Such softmax objectives have been shown to outperform binary cross-entropy with few negatives for sequential recommenders like SASRec [14].

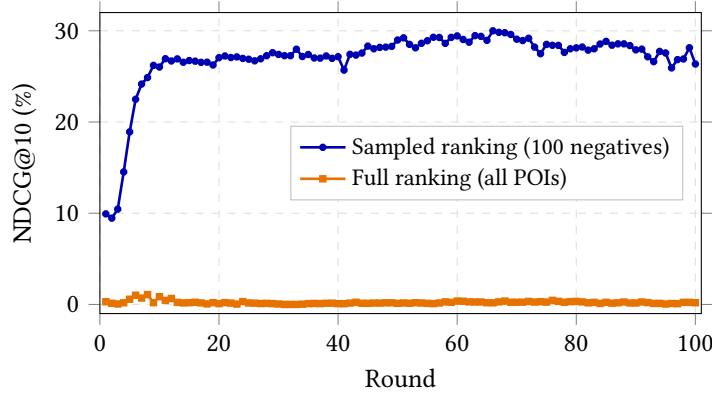


Fig. 3. Per round test score on the Foursquare NYC dataset, showing NDCG@10 over communication rounds under sampled ranking (blue) and full ranking (orange). Switching from sampled to full ranking causes the original FedKG implementation score to collapse from $\approx 30\%$ to $\approx 1\%$, revealing that the sampled metric vastly overstates performance.

3.5.3 Stabilising Federated Optimization. Federated training is inherently fragile so a poorly tuned optimizer can easily cause the model to diverge or overfit. To address this we introduced those changes:

- *Learning rate.* The fixed large rate of 0.01 can cause the training to overshoot. We reduced it to 0.001 and combined it with warmup (small initial steps) and cosine decay (small final steps)
- *Regularization.* To prevent the model from memorizing its few check-ins (overfitting), we added weight decay and an L_2 penalty, which keep the parameters small and apply early stopping once the validation score plateaus
- *Model capacity.* The original configuration (50-dimensional embeddings, one attention block) we enlarged to 64 dimensions and two blocks because the full ranking task is harder.
- *Client participation.* The original join ratio of 0.4 trains only 40 % of the clients per round. With only five clients this waste scarce data, so we raised it to 1.0 such that every client contributes in every round.

4 System Implementation and Frontend Prototype

During our work, we have focused on the backend and decentralized recommendation models; we will keep the frontend documentation short and focus only on high-level decisions. We have an extensive documentation with screenshots of the frontend prototype on our GitHub repository; you can find it here¹.

4.1 Interactive Map-Based Recommender System

For the recommendation process, there are several ways to proceed. For example, recommendations could appear next to a planned path [19], or in a list like on dating platforms. As described in Section 1, the goal is to proactively notify the user of interesting places in their vicinity. Therefore, we decided to recommend places based on the tourist’s current location (Section 3.3.2). We introduce a map that marks the user’s location and highlights nearby points of interest. Based on our experiments in Section 5, we decided that recommending ten places is a good tradeoff between usability (that the screen is not too crowded) and hit rate (that the actual check-in of the user is among the recommendations).

¹github.com/projects-in-recsys-ss26/w2-decentralized-recommender-system

To make the prototype feel realistic, we feed the models with actual user data from the Foursquare dataset and display the recommendations of our two models based on the constructed input data of concrete sample users. With a button, we can click through every user in the dataset and visualize their recommendations.

In the category gossip model, based on a location, user group, and time, we predict five specific categories for the next PoI. For the user group, we build the user features for the current user and calculate the distance to the trained centroids. For the simulated location, we use the user's last check-in time. For the location, we obfuscate the last check-in location within a 400m radius. Since we want to display concrete locations in the vicinity rather than categories from the model, we recommend selecting the two closest matching places per category based on the simulated location.

For the FL model, we use all past check-ins for the current user to predict the ten most likely check-ins and display them as recommendations.

4.2 Proactive Contextual Notifications

Because the user should not only receive recommendations when they ask for them but proactively (Section 1), we implemented notifications on the lock screen and in the unlocked map view. When a user is close to PoI with a very high predicted rating, the app can send a notification to the user's device. With that, they get proactively informed about interesting PoIs without even opening the app.

4.3 Transparent Privacy Controls

We trained two models with different feature spaces and different performances. With that, they rely on different inputs. The category gossip model (see Section 3.3.3) relies only on the location's categories and time, but performs worse. The FL approach relies on the full check-in history but achieves a significantly higher PoI-hit rate. Lastly, the feature that proactively notifies the user about interesting places in the vicinity requires steady access to the user's location, even when the app is closed.

We let the user decide which permissions he wants to grant to our prototype, and based on that, decide which model to use. When the user does not grant permission for a feature, we notify them of how the decision affects the recommender system's performance. You can see the settings and how turning them off warns the user in the following:

- *Background Location*: Disabling background location access prevents the app from sending timely recommendations and notifications while not actively used.
- *Share Exact Check-ins*: Does not prevent notifications, but may significantly reduce the accuracy and relevance of personalized recommendations.
- *Personal Information*: Limits the ability to improve recommendation algorithms for the broader community. However, personal recommendation quality remains unaffected.

The "background location" setting would transform our app from a proactive recommender system to a demand-driven one. The setting that disables sharing exact check-ins switches the used model from the FL to the category gossip model. If the user agrees to share their user data, we can train the user model. If not, we leave the user out of the gossip in Section 3.3.3, the user's performance does not worsen. In the description of the setting, we also added age, gender, and related information to the data we share. These features could easily be integrated into our user model, but because the dataset we use (see Section 3.2) does not include this information, we could not train on it.

5 Results

5.1 Experimental Setup

For all experiments, we adopt a leave-one-out evaluation strategy. We train the models on all but the last check-in of each user, using this final datapoint as the ground truth for testing. Based on the user’s history, the models predict the next k PoIs (or categories). A recommendation is considered successful if the actual next check-in falls within these top k predictions. The percentage of successful recommendations across all test cases is reported as *HitRate@k* [13].

5.2 Category-based Performance

The category-gossip model predicts the next PoI’s category. We evaluate the *HitRate@5* and simulate scenarios where varying numbers of users participate in the training set (see Fig. 4).

As expected, the centralized cluster-based baseline performs best with a *HitRate@5* of 40.05%, since the decentralized versions add Laplacian noise to preserve privacy during peer data exchange. However, performance heavily depends on the number of gossip rounds: while 5 rounds yield only 23.2%, increasing the exchange to 10 and 20 rounds improves the *HitRate@5* to 35% and 39%, respectively. Given that the time-based baseline by Wiedemann et al. [32] achieves a *HitRate@1* of 29.7%, our results are highly competitive. Most notably, with sufficient gossip rounds, the decentralized privacy-preserving model successfully converges to a performance nearly identical to the centralized baseline.

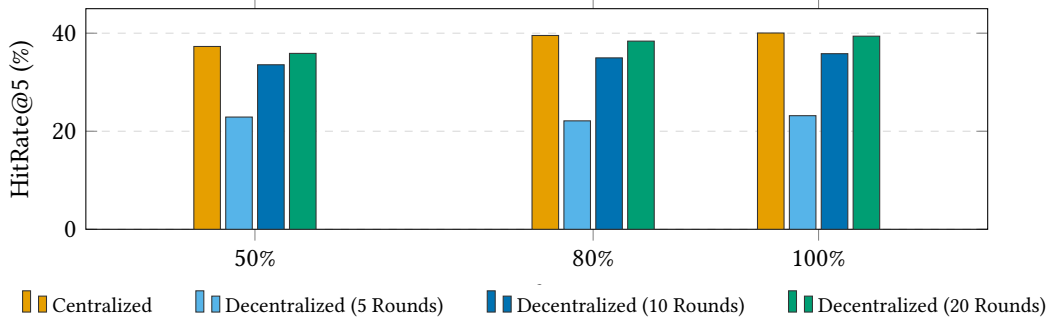


Fig. 4. *HitRate@5* of different models predicting the most specific category of the last checkin available for each user, evaluated on 50%, 80%, and 100% of the training set. Decentralized models differ by the number of gossip rounds they are trained with.

5.3 FedKG Performance

Fig. 5 and Fig. 6 show the test *NDCG@10* and *HR@10* under full ranking over 100 communication rounds on Foursquare NYC with and without the ST-GAN augmentation, all other settings are identical. Both runs train stably, after a brief dip in the earliest rounds the curves rise smoothly, confirming that the optimization changes of Section 3.5.3 work as intended. The difference between the variants however is large. Without the GAN the model improves steadily to 27.0% *NDCG@10* and 32.2% *HR@10*, while with the GAN it plateaus from about round 75 at 17.2% and 20.6%. In sum the honest evaluation and the aligned training objective recover usable performance from $\approx 1\%$ *NDCG@10* in the original setup to 27.0% with our tuning. The likely cause is that the ST-GAN fails to synthesise sufficiently realistic trajectories. Instead of restoring the disrupted spatial patterns, its samples introduce noise into training. The choice between the variants is thus a privacy performance trade-off.

Without the GAN the recommendation quality is substantially higher. With the GAN training on a mixture of real and synthetic trajectories obfuscates individual check-in histories and hardens the model against reconstruction attacks. Which variant to deploy depends on the privacy requirements of the application.

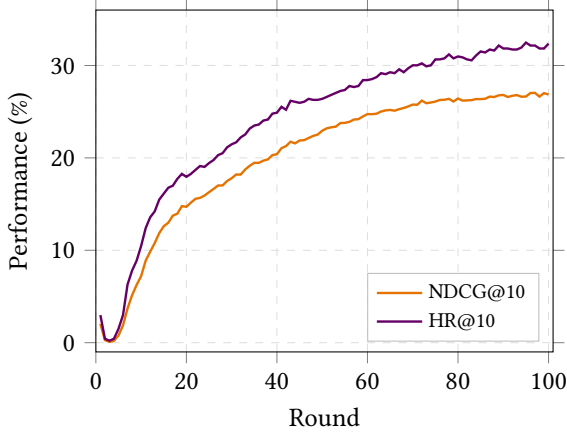


Fig. 5. Tuned FedKG without ST-GAN on Foursquare NYC: per-round test metrics under full ranking. Training converges smoothly to a best NDCG@10 of 27.0 % (HR@10 = 32.2 %).

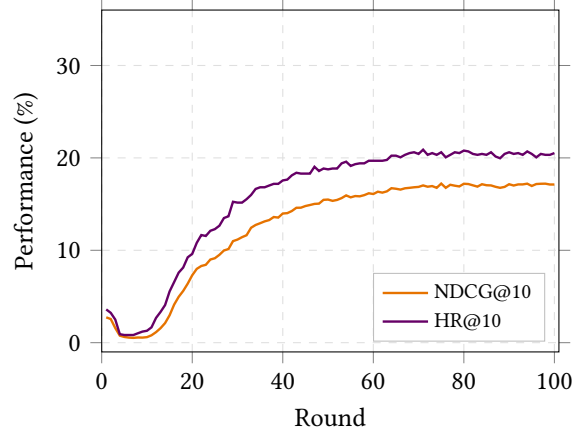


Fig. 6. Tuned FedKG with ST-GAN, identical settings otherwise. The model plateaus from about round 75 at a best NDCG@10 of 17.2 % (HR@10 = 20.6 %), roughly ten percentage points below the GAN-free variant.

6 Limitations and Future Work

We used the Foursquare dataset for training our models. Unfortunately, they are constrained to places in Tokyo and New York City. Therefore, our tourist recommender system is currently constrained to predicting places in those two cities. Future work could focus on creating a dataset following the Foursquare structure in other cities.

In the category gossip model, we predict five specific categories for the next PoI. Since our recommender system needs to predict concrete locations in the vicinity, we needed a method to convert the predicted categories into locations we can display in the frontend prototype. For that, we recommended the two closest matching places per category. With varying simulated locations, the performance of predicting exact locations was poor. Therefore, in future work, implementing a more dedicated method for selecting places in the vicinity with a matching category could be of interest.

The transformer-based approach has two main limitations. First SASRec uses only the sequence of POI IDs, coordinates, timestamps, and venue categories are available but ignored, although they carry strong predictive value. This follows the original FedKG setup and keeps the model lightweight enough to train on end devices. Second all experiments use only five clients, also following the original configuration. Future work could look into whether it is possible to include this feature and evaluate at larger client counts. On the application side, recommendations could be filtered by opening hours and vicinity and the system deployed on real mobile devices.

7 Conclusion

In this master project, we demonstrated that decentralized next PoI recommender architectures can achieve near-centralized performance while preserving users' privacy. Using approaches from decentralized and device-cloud collaborative learning, we achieved hit rates similar to those of centralized systems in New York City. In the future, we could extend the domain to more cities and bridge the gap between category and PoI prediction.

8 GenAI disclosure

For code generation, we have used Gemini 3.1 Pro², Claude Opus 4.6³, Claude Opus 4.8⁴ and Claude Fable 5⁵. For the report, for spelling or grammar mistakes, we used Grammarly⁶. To our knowledge, this is not AI but rule-based; we want to list it here anyway in case it is.

9 List of Abbreviations

FL	Federated Learning
GKD	Global Knowledge Distillation
LSBNs	Location-Based Social Networks
NYC	New York City
PKD	Personalized Knowledge Distillation
PoI	Point-of-Interest
RecSys	Recommender System
SASRec	Self-Attentive Sequential Recommendation
TKY	Tokyo

References

- [1] Hong-Kyun Bae, Yebeen Kim, Hyunjoon Kim, and Sang-Wook Kim. 2024. Negative Sampling in Next-POI Recommendations: Observation, Approach, and Evaluation. In *Proceedings of the ACM Web Conference 2024 (WWW '24)*. Association for Computing Machinery, New York, NY, USA, 3888–3899. doi:10.1145/3589334.3645681
- [2] Yacine Belal, Aurélien Bellet, Sonia Ben Mokhtar, and Vlad Nitu. 2022. PEPPER: Empowering User-Centric Recommender Systems over Gossip Learning. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 6, 3, Article 101 (Sept. 2022), 27 pages. doi:10.1145/3550302
- [3] Qiqi Cai, Jian Cao, Guandong Xu, and Nengjun Zhu. 2024. Distributed Recommendation Systems: Survey and Research Directions. *ACM Trans. Inf. Syst.* 43, 1 (Nov. 2024), 10:1–10:38. doi:10.1145/3694783
- [4] Jie Chen and Ronny Luss. 2019. Stochastic Gradient Descent with Biased but Consistent Gradient Estimators. doi:10.48550/arXiv.1807.11880 arXiv:1807.11880 [cs.LG].
- [5] Chen Cheng, Haiqin Yang, Michael R. Lyu, and Irwin King. 2013. Where you like to go next: successive point-of-interest recommendation. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI '13)*. AAAI Press, 2605–2611.
- [6] Yan Cui. 2015. An Evaluation of Yelp Dataset. *CoRR* abs/1512.06915 (2015). <http://arxiv.org/abs/1512.06915> arXiv: 1512.06915.
- [7] Edwige Cyffers, Mathieu Even, Aurélien Bellet, and Laurent Massoulié. 2022. Muffliato: Peer-to-Peer Privacy Amplification for Decentralized Optimization and Averaging. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 15889–15902. https://proceedings.neurips.cc/paper_files/paper/2022/file/65d32185f73cbf4535449a792c63926f-Paper-Conference.pdf
- [8] Jérôme Fellus, David Picard, and Philippe-Henri Gosselin. 2013. Decentralized K-Means Using Randomized Gossip Protocols for Clustering Large Datasets. In *2013 IEEE 13th International Conference on Data Mining Workshops*. 599–606. doi:10.1109/ICDMW.2013.58
- [9] Huiji Gao, Jiliang Tang, Xia Hu, and Huan Liu. 2013. Exploring temporal effects for location recommendation on location-based social networks. In *Proceedings of the 7th ACM conference on Recommender systems*. ACM, Hong Kong China, 93–100. doi:10.1145/2507157.2507182
- [10] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative Adversarial Networks. <https://arxiv.org/abs/1406.2661> _eprint: 1406.2661.
- [11] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. <https://arxiv.org/abs/1511.06939> _eprint: 1511.06939.
- [12] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531 [stat.ML] <https://arxiv.org/abs/1503.02531>

²docs.cloud.google.com/gemini-enterprise-agent-platform/models/gemini/3-1-pro

³anthropic.com/news/claude-opus-4-6

⁴anthropic.com/news/claude-opus-4-8

⁵anthropic.com/news/claude-fable-5

⁶grammarly.com

- [13] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. doi:10.1109/ICDM.2018.00035
- [14] Anton Klenitskiy and Alexey Vasilev. 2023. Turning Dross Into Gold Loss: is BERT4Rec really better than SASRec?. In *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys '23)*. ACM, 1120–1125. doi:10.1145/3604915.3610644
- [15] Anastasia Koloskova, Nicolas Loizou, Sadra Boreiri, Martin Jaggi, and Sebastian Stich. 2020. A Unified Theory of Decentralized SGD with Changing Topology and Local Updates. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*, Hal Daumé III and Aarti Singh (Eds.). PMLR, 5381–5393. <https://proceedings.mlr.press/v119/koloskova20a.html>
- [16] Walid Krichene and Steffen Rendle. 2022. On sampled metrics for item recommendation. *Commun. ACM* 65, 7 (June 2022), 75–83. doi:10.1145/3535335
- [17] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*. Association for Computing Machinery, New York, NY, USA, 1269–1278. doi:10.1145/3292500.3330895
- [18] Nicholas Lim, Bryan Hooi, See-Kiong Ng, Xueou Wang, Yong Liang Goh, Renrong Weng, and Jagannadan Varadarajan. 2020. STP-UDGAT: Spatial-Temporal-Preference User Dimensional Graph Attention Network for Next POI Recommendation. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (CIKM '20)*. Association for Computing Machinery, New York, NY, USA, 845–854. doi:10.1145/3340531.3411876
- [19] Yan Lin, Huaiyu Wan, Shengnan Guo, and Youfang Lin. 2021. Pre-training Context and Time Aware Location Embeddings from Spatial-Temporal Trajectories for User Next Location Prediction. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 5 (May 2021), 4241–4248. doi:10.1609/aaai.v35i5.16548
- [20] Tian Liu, Yue Cui, Xueyang Hu, Yecheng Xu, and Bo Liu. 2024. Technical Report: On the Convergence of Gossip Learning in the Presence of Node Inaccessibility. doi:10.48550/arXiv.2401.09498 arXiv:2401.09498 [cs.LG].
- [21] Stuart Lloyd. 1982. Least squares quantization in PCM. *IEEE transactions on information theory* 28, 2 (1982), 129–137.
- [22] Jing Long, Guanhua Ye, Tong Chen, Yang Wang, Meng Wang, and Hongzhi Yin. 2024. Diffusion-Based Cloud-Edge-Device Collaborative Learning for Next POI Recommendations. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 2026–2036. doi:10.1145/3637528.3671743
- [23] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2023. Communication-Efficient Learning of Deep Networks from Decentralized Data. <https://arxiv.org/abs/1602.05629> _eprint: 1602.05629.
- [24] Henrik Mühe, Wolfgang Wörndl, and Georg Groh. 2009. Realisierung eines dezentralen Recommender Systems für PDAs. In *Proceedings der 4. Konferenz Mobilität und mobile Informationssysteme (MMS 2009)*. Münster, Germany.
- [25] Shengyang Qin, Xinyu Lei, Nankun Mu, Hongyu Huang, Tian Xie, and Xiao Zhang. 2026. Uncovering Risks of Data-Free Feature Vector Inversion Attacks Against Vector Databases. *IEEE Transactions on Dependable and Secure Computing* 23, 1 (2026), 193–208. doi:10.1109/TDSC.2025.3605268
- [26] Agnishwar Raychaudhuri. 2025. The Facebook-Cambridge Analytica Data Scandal: A Case Study in Ethical AI Failures and Data Privacy Violations. *IRE Journals* 9, 4 (Oct. 2025), 1531–1535. doi:10.64388/IREV9I4-1711588-7074
- [27] Zehua Sun, Yonghui Xu, Yong Liu, Wei He, Lanju Kong, Fangzhao Wu, Yali Jiang, and Lizhen Cui. 2025. A Survey on Federated Recommendation Systems. *IEEE Transactions on Neural Networks and Learning Systems* 36, 1 (2025), 6–20. doi:10.1109/TNNLS.2024.3354924
- [28] Jiayi Tang and Ke Wang. 2018. Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding. <https://arxiv.org/abs/1809.07426> _eprint: 1809.07426.
- [29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. <https://arxiv.org/abs/1706.03762> _eprint: 1706.03762.
- [30] Li Wan, Wee Keong Ng, Shuguo Han, and Vincent C. S. Lee. 2007. Privacy-preservation for gradient descent methods. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '07)*. Association for Computing Machinery, New York, NY, USA, 775–783. doi:10.1145/1281192.1281275
- [31] Bin Wang, Yan Zhang, Yan Ma, Yaohui Jin, and Yanyan Xu. 2024. SA-LSPL: Sequence-Aware Long- and Short- Term Preference Learning for next POI recommendation. *ArXiv abs/2404.00367* (2024). <https://api.semanticscholar.org/CorpusID:268819275>
- [32] Nina Wiedemann, Krzysztof Janowicz, Martin Raubal, and Ourania Kounadi. 2024. Where you go is who you are: a study on machine learning based semantic privacy attacks. *Journal of Big Data* 11, 1 (March 2024), 39. doi:10.1186/s40537-024-00888-8
- [33] Wilson Wongso, Hao Xue, and Flora D. Salim. 2025. GenUP: Generative User Profilers as In-Context Learners for Next POI Recommender Systems. <https://arxiv.org/abs/2410.20643> _eprint: 2410.20643.
- [34] Huaming Wu, Zirui Zhang, Chang Guan, Katinka Wolter, and Minxian Xu. 2020. Collaborate Edge and Cloud Computing With Distributed Deep Learning for Smart City Internet of Things. *IEEE Internet of Things Journal* 7, 9 (2020), 8099–8110. doi:10.1109/JIOT.2020.2996784
- [35] Jiancan Wu, Xiang Wang, Xingyu Gao, Jiawei Chen, Hongcheng Fu, and Tianyu Qiu. 2024. On the Effectiveness of Sampled Softmax Loss for Item Recommendation. *ACM Trans. Inf. Syst.* 42, 4, Article 98 (March 2024), 26 pages. doi:10.1145/3637061

- [36] Dingqi Yang, Daqing Zhang, Vincent W. Zheng, and Zhiyong Yu. 2015. Modeling User Activity Preference by Leveraging User Spatial Temporal Characteristics in LBSNs. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 45, 1 (Jan. 2015), 129–142. doi:10.1109/TSMC.2014.2327053
- [37] Jiangchao Yao, Feng Wang, KunYang Jia, Bo Han, Jingren Zhou, and Hongxia Yang. 2021. Device-Cloud Collaborative Learning for Recommendation. doi:10.48550/arXiv.2104.06624 arXiv:2104.06624 [cs.LG].
- [38] Selim F. Yilmaz, Burak Hasircioglu, Li Qiao, and Deniz Gunduz. 2025. Private Collaborative Edge Inference via Over-the-Air Computation. *IEEE Transactions on Machine Learning in Communications and Networking* 3 (2025), 215–231. doi:10.1109/TMLCN.2025.3526551 arXiv:2407.21151 [cs.LG].
- [39] Dongxiao Yu, Zhenzhen Xie, Yuan Yuan, Shuzhen Chen, Jing Qiao, Yangyang Wang, Yong Yu, Yifei Zou, and Xiao Zhang. 2023. Trustworthy decentralized collaborative learning for edge intelligence: A survey. *High-Confidence Computing* 3, 3 (2023), 100150. doi:10.1016/j.hcc.2023.100150
- [40] Jun Zeng, Yizhu Zhao, Ziwei Wang, Hongjin Tao, Min Gao, and Junhao Wen. 2023. LGSA: A next POI prediction method by using local and global interest with spatiotemporal awareness. *Expert Systems with Applications* 227 (Oct. 2023), 120291. doi:10.1016/j.eswa.2023.120291
- [41] Kangzhi Zhao, Yong Zhang, Hongzhi Yin, Jin Wang, Kai Zheng, Xiaofang Zhou, and Chunxiao Xing. 2021. Discovering subsequence patterns for next POI recommendation. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI'20)*.
- [42] Lin Zhong, Jun Zeng, Ziwei Wang, Haoran Tang, and Junhao Wen. 2026. FedKG: Federated POI recommendation via generative knowledge distillation against spatial heterogeneity. *Expert Systems with Applications* 304 (April 2026), 130652. doi:10.1016/j.eswa.2025.130652

A Additional Figures

A.1 Related Work

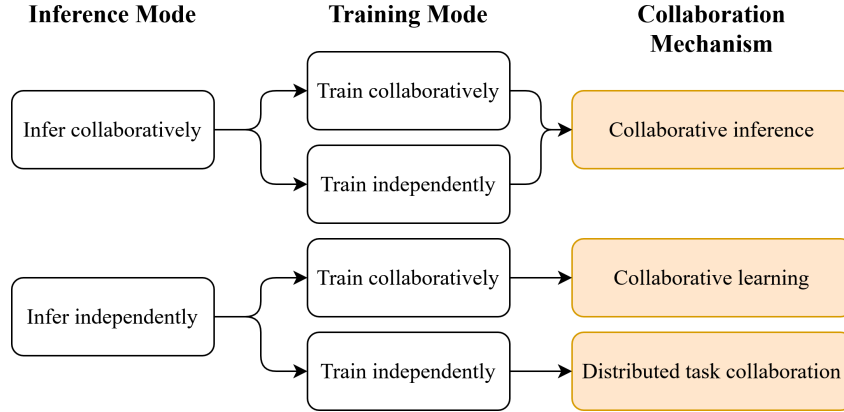


Fig. 7. Categorization of different distributed recommender systems as in [3]. Based on the inference and the training happening independently or collaboratively, they categorize the method as collaborative inference, collaborative learning, or distributed task collaboration.

A.2 Interesting User Clusters

In the following, you can see three interesting user clusters which are entirely learned from the user data, without any manual coding. You can see that three, possibly more, of the user cluster centroids actually could be tourists.

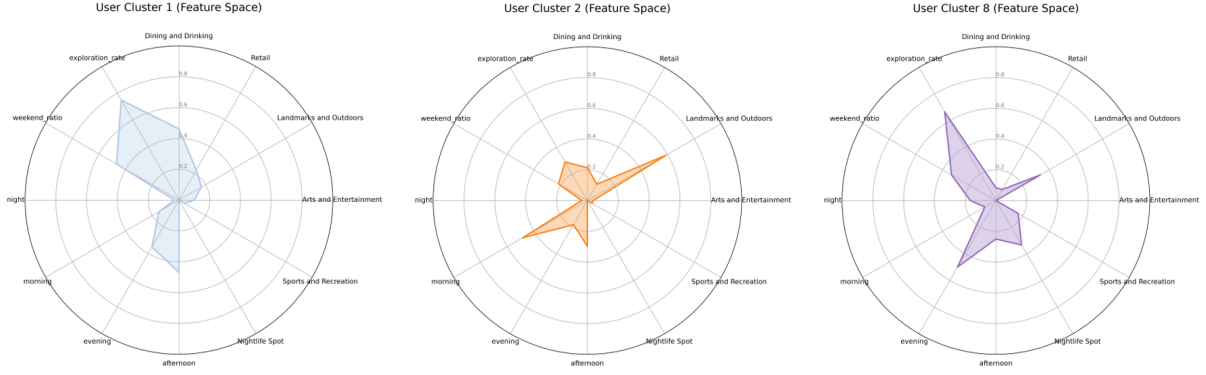


Fig. 8. Three interesting user clusters: Classic tourist, Outdoor tourist, and Nightlife tourist.

B Pseudocodes to decentralized category gossip models

B.1 Decentralized Gossip K-Means

Algorithm 1 Decentralized Gossip K-Means

Require: Network of nodes, local feature X_i for node i , clusters k , epochs E , gossip rounds R

```

1: // Phase 1: Decentralized Standardization
2: Initialize local sums  $S_i \leftarrow X_i$  and squared sums  $S_i^2 \leftarrow X_i^2$ 
3: for  $r = 1$  to  $R$  do
4:   Connect random pairs of nodes  $(i, j)$ 
5:   Exchange and average:  $S_i, S_j \leftarrow \frac{S_i + S_j}{2}$  and  $S_i^2, S_j^2 \leftarrow \frac{S_i^2 + S_j^2}{2}$ 
6: end for
7: Compute global  $\mu, \sigma$  locally from  $S_i, S_i^2$  and scale data:  $X_i \leftarrow \frac{X_i - \mu}{\sigma}$ 
8: // Phase 2: Decentralized K-Means
9: Initialize  $k$  centroids  $C = \{c_1, \dots, c_k\}$  (e.g., via a shared random seed)
10: for  $epoch = 1$  to  $E$  do
11:   // 1. Local Assignment
12:   Find nearest centroid:  $c^* \leftarrow \arg \min_{c \in C} \|X_i - c\|$ 
13:   Set local sum  $sum_i[c^*] \leftarrow X_i$  (otherwise 0 for all other  $c$ )
14:   Set local count  $count_i[c^*] \leftarrow 1$  (otherwise 0 for all other  $c$ )
15:   // 2. Gossip for Centroids
16:   for  $r = 1$  to  $R$  do
17:     Connect random pairs of nodes  $(i, j)$ 
18:     Exchange and average for all  $c \in C$ :
19:      $sum_i[c], sum_j[c] \leftarrow \frac{sum_i[c] + sum_j[c]}{2}$ 
20:      $count_i[c], count_j[c] \leftarrow \frac{count_i[c] + count_j[c]}{2}$ 
21:   end for
22:   // 3. Update Centroids
23:    $c \leftarrow \frac{sum_i[c]}{count_i[c]}$  for all  $c \in C$ 
24: end for

```

B.2 Decentralized Recommender System

Algorithm 2 Decentralized Recommender with Gossip & Local Differential Privacy (LDP)

Require: Set of nodes (users) V , local check-ins D_i for each node i , privacy budget ϵ , gossip rounds R , top recommendations k

```

1: // Phase 1: Local Computation & LDP Noise Addition
2: for each node  $i \in V$  do
3:   Calculate local frequency histograms  $H_i$  from local check-ins  $D_i$ 
4:   //  $H_i$  contains global counts, hourly counts, and optionally cluster-specific counts
5:   Initialize noisy histograms  $\tilde{H}_i \leftarrow \emptyset$ 
6:   for each category count  $c \in H_i$  do
7:     Sample noise  $w \sim \text{Laplace}(0, 1/\epsilon)$ 
8:      $\tilde{H}_i[c] \leftarrow \max(0, c + w)$  // Add noise and clip to zero
9:   end for
10: end for
11: // Phase 2: Decentralized Gossip Averaging
12: for  $r = 1$  to  $R$  do
13:   Randomly shuffle nodes and group them into disjoint pairs  $(i, j)$ 
14:   for each pair  $(i, j)$  do
15:     // Exchange dictionaries and compute average for every key
16:      $\tilde{H}_i, \tilde{H}_j \leftarrow \frac{\tilde{H}_i + \tilde{H}_j}{2}$ 
17:   end for
18: end for
19: // Phase 3: Top-K Extraction (Post-Convergence)
20: // After  $R$  rounds, all nodes hold approximately the global average histogram
21: Let  $\tilde{H} \leftarrow \tilde{H}_0$  // Take histogram from any representative node
22: for each time context  $t$  (e.g., hour, cluster) in  $\tilde{H}$  do
23:   Sort categories in  $\tilde{H}[t]$  by count in descending order
24:   Extract top- $k$  categories for context  $t$ 
25: end for

```
